MOTORBIKE DETECTION IN FIXED-CAMERA TRAFFIC VIDEO WITH YOLOv8: AN IMAGE-MINING PIPELINE FROM DATA AUDITING TO ERROR ANALYSIS

Trần Hoàng Đạt[1]
Personal self-study project report (not a journal submission)

REPORT INFO

Started: 07/10/2026

Completed: 08/10/2026

Version: v2

KEYWORDS

Object detection
YOLOv8
Data leakage
Video-based split
Error analysis

ABSTRACT

This self-study report documents an end-to-end image-mining pipeline for detecting motorbikes in fixed-camera traffic video with YOLOv8s. The public Roboflow dataset holds 1,355 frames from 13 videos (1,152 train, 136 validation, 67 unlabeled test). Before training, the data were audited with basic image processing: brightness, contrast and Laplacian-variance sharpness, label validation, and a 64-bit difference hash to find duplicates. The audit found 45 groups of identical images, 12 of them crossing split boundaries. Two datasets were then compared under identical settings: A, the original leaky split, and B, a de-duplicated split that holds out whole videos. On two unseen videos, B reached mAP@0.5 = 0.980 and mAP@0.5:0.95 = 0.775, versus 0.990 and 0.806 for A, so leakage inflated the scores only slightly. At the F1-optimal threshold the model made 355 correct detections, 85 false alarms and 14 misses; most false alarms turned out to be real motorbikes missing from the labels. Key limitations are incomplete labels, a single scene type, a two-video validation set and no temporal analysis.

PHÁT HIỆN XE MÁY TRONG VIDEO GIAO THÔNG BẰNG YOLOv8: QUY TRÌNH KHAI PHÁ DỮ LIỆU ẢNH TỪ KIỂM TRA DỮ LIỆU ĐẾN PHÂN TÍCH LỖI

Trần Hoàng Đạt
Báo cáo đồ án tự luyện cá nhân (không phải bài báo nộp tạp chí)

THÔNG TIN BÁO CÁO

Ngày bắt đầu: 07/10/2026

Ngày hoàn thành: 08/10/2026

Phiên bản: v2

TỪ KHÓA

Phát hiện đối tượng

TÓM TẮT

Báo cáo trình bày trọn một quy trình khai phá dữ liệu ảnh để phát hiện xe máy trong video giao thông quay bằng camera cố định, dùng mô hình YOLOv8s. Bộ dữ liệu công khai trên Roboflow gồm 1.355 khung hình từ 13 video (1.152 ảnh huấn luyện, 136 ảnh kiểm định, 67 ảnh kiểm thử không nhãn). Trước khi huấn luyện, dữ liệu được kiểm tra bằng xử lý ảnh cơ bản: thống kê độ sáng, độ tương phản, độ nét theo phương sai Laplacian, kiểm tra nhãn và mã băm sai khác 64 bit để tìm ảnh trùng. Kết quả có 45 nhóm ảnh trùng hệt nhau, 12 nhóm nằm vắt qua các tập. Hai phiên bản dữ liệu được so sánh trong cùng điều kiện: bản A giữ cách chia gốc bị rò rỉ, bản B loại trùng và tách riêng trọn

[1]Tác giả liên hệ (corresponding author). *Email: hoangdattran12@gmail.com*

YOLOv8
Rò rỉ dữ liệu
Chia dữ liệu theo video
Phân tích lỗi

video để kiểm định. Trên hai video chưa từng thấy, bản B đạt mAP@0.5 bằng 0,980 và mAP@0.5:0.95 bằng 0,775, so với 0,990 và 0,806 của bản A, tức rò rỉ chỉ làm điểm tăng nhẹ. Ở ngưỡng tối ưu theo F1, mô hình có 355 dự đoán đúng, 85 báo nhầm và 14 bỏ sót; phần lớn báo nhầm thực chất là xe máy có thật bị bỏ sót khi gán nhãn. Hạn chế chính là nhãn chưa đầy đủ, chỉ một kiểu khung cảnh, tập kiểm định chỉ hai video và chưa khai thác yếu tố thời gian.

1. Giới thiệu

1.1. Bối cảnh và bài toán

Xe máy là phương tiện chiếm đa số trên đường phố Việt Nam. Nếu máy tính tự động **tìm được vị trí từng chiếc xe máy** trong khung hình camera, ta có thể xây dựng tiếp các ứng dụng như đếm lưu lượng, đo mật độ theo giờ hay phát hiện ùn tắc. Bài toán nền tảng ở đây là **phát hiện đối tượng** (object detection): với mỗi ảnh, mô hình phải trả về danh sách các hình chữ nhật bao quanh vật thể (bounding box, gọi tắt là *box*) cùng với một điểm tin cậy cho biết mô hình chắc chắn đến đâu.

Bài toán này khác với **phân loại ảnh** đã học trong học máy cơ bản ở hai điểm. Thứ nhất, số đối tượng trong một ảnh không cố định, có ảnh không có xe, có ảnh có năm xe. Thứ hai, ngoài việc trả lời "có phải xe máy không" còn phải trả lời "xe nằm ở đâu", tức là một bài toán hồi quy toạ độ đi kèm. Vì vậy, cách đánh giá cũng khác: không chỉ đếm đúng/sai mà còn phải đo xem box dự đoán khớp với box thật đến mức nào (mục 2.5).

Báo cáo này coi toàn bộ công việc như một đồ án **khai phá dữ liệu ảnh** (image mining) theo quy trình khám phá tri thức (KDD) [1] và CRISP-DM [2]: hiểu dữ liệu, chuẩn bị dữ liệu, mô hình hoá, đánh giá và rút ra tri thức. Mô hình được chọn là YOLOv8s, một phiên bản của họ YOLO [3] do Ultralytics phát triển [4]; dữ liệu là bộ ảnh công khai trên Roboflow [5].

1.2. Mục tiêu và câu hỏi khai phá

Đồ án đặt ra ba câu hỏi, mỗi câu tương ứng với một phần kết quả:

- **CH1 – Dữ liệu có vấn đề gì?** Nhãn có hợp lệ không, ảnh có trùng lặp không, cách chia train/valid có gây rò rỉ không (mục 3.2–3.5).
- **CH2 – Mô hình hoạt động tốt đến đâu trên video chưa từng thấy?** So sánh hai cách chia dữ liệu A và B trong cùng điều kiện huấn luyện (mục 4.1–4.3).
- **CH3 – Lỗi còn lại đến từ đâu?** Lỗi do mô hình hay do dữ liệu; loại xe nào hay bị bỏ sót (mục 4.4–4.6).

1.3. Cách đọc báo cáo cùng với mã nguồn

Báo cáo được viết để đọc song song với hai notebook: `motorbike-detection-english.ipynb` (bản chạy thật trên Kaggle) và `motorbike_detection-viet.ipynb` (bản tiếng Việt để đối chiếu, cùng cấu trúc từng ô). Bảng 1 cho biết mỗi mục của báo cáo ứng với mục nào (cột §) và hàm nào trong notebook. Ký hiệu "§" trong báo cáo luôn chỉ số mục của notebook.

Bảng 1. *Đối chiếu báo cáo với notebook*

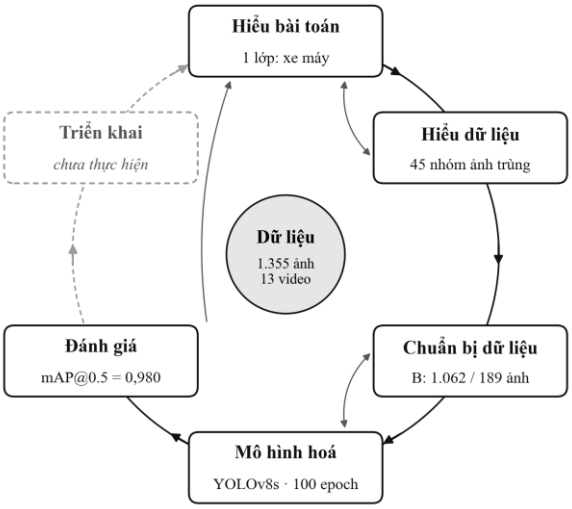
Mục báo cáo	§	Hàm / biến chính
3.1 Dữ liệu	§2	`list_images`, `video_id`, `meta`
3.2 Khảo sát nhãn	§3	`read_labels`, `boxes`
3.3 Chất lượng ảnh	§4.1	`img_stats`
3.3 Kiểm tra nhãn	§4.2	`iou_xywh`, `issues`
3.4 Trùng lặp, rò rỉ	§4.3	`dhash`, `find`, `hamming`
3.5 Tiền xử lý	§5.1	`clean_labels`, `build_dataset`, `GroupShuffleSplit`
3.6 Tăng cường	§5.2	`AUG`
3.7 Huấn luyện	§6	`train_run`
4.1–4.2 Kết quả	§7.1–7.2	`hist`, `score`
4.3 Ngưỡng	§7.3	`BEST_CONF`
4.4 Phân tích lỗi	§7.4	`iou_matrix`, `err`, `pim`
4.5 Suy luận	§8–9	`pred_df`, xuất ONNX

2. Cơ sở lý thuyết và lý do lựa chọn kỹ thuật

2.1. Quy trình khai phá dữ liệu ảnh

Khai phá dữ liệu không bắt đầu bằng việc huấn luyện mô hình mà bằng việc **hiểu dữ liệu**. Đồ án đi theo chu trình CRISP-DM [2] (Hình 1), gồm sáu pha quanh một trung tâm là dữ liệu. Mỗi ô trong hình ghi một con số thật tiêu biểu cho pha đó. Đồ án đã đi qua năm pha đầu: hiểu bài toán (§1), hiểu dữ liệu (§2–§4), chuẩn bị dữ liệu (§5), mô hình hoá (§6) và đánh giá (§7–§9). Pha **triển khai** (đưa mô hình vào chạy trên camera thật) **chưa được thực hiện** trong báo cáo này, nên được vẽ bằng nét đứt.

Các mũi tên bên trong vòng là những bước **quay lui** đặc trưng của CRISP-DM, và đồ án này đã gặp đúng các bước đó. Khi hiểu dữ liệu, việc phát hiện ảnh trùng làm thay đổi cách đặt bài toán đánh giá. Khi mô hình hoá, kết quả huấn luyện buộc phải quay lại phần chuẩn bị dữ liệu. Kết quả đánh giá (nhãn bị thiếu) lại đặt ra câu hỏi mới cho chính bài toán. Ở pha chuẩn bị, dữ liệu được tách thành **hai phiên bản A và B** để làm một thí nghiệm đối chứng: hai lần huấn luyện dùng cùng mô hình, cùng siêu tham số, cùng phép tăng cường, chỉ khác dữ liệu. Nhờ vậy, mọi chênh lệch điểm số có thể quy về sự khác nhau của dữ liệu.



Hình 1. *Chu trình CRISP-DM áp dụng cho đồ án. Mỗi ô ghi một số liệu thật của pha tương ứng: 1 lớp đối tượng; 45 nhóm ảnh trùng phát hiện được; bản B có 1.062 ảnh train và 189 ảnh val; YOLOv8s huấn luyện tối đa 100 epoch; mAP@0.5 = 0,980 trên val của B. Vòng ngoài là thứ tự các pha; mũi tên trong là các bước quay lui; nét đứt xám là pha triển khai, chưa thực hiện trong báo cáo này*

2.2. Các họ thuật toán phát hiện đối tượng và lý do chọn YOLO

Có bốn hướng tiếp cận chính, được tóm tắt trong Bảng 2.

- **Phương pháp cổ điển** như Viola–Jones [6] hay HOG + SVM [7]: trích đặc trưng thủ công (biên cạnh, gradient) rồi cho bộ phân loại quét qua cửa sổ trượt. Nhanh và dễ hiểu, nhưng đặc trưng thủ công kém bền khi ánh sáng, góc nhìn và kiểu dáng thay đổi.
- **Hai giai đoạn (two-stage)** như Faster R-CNN [8]: giai đoạn 1 đề xuất các vùng có thể chứa vật, giai đoạn 2 phân loại và chỉnh box cho từng vùng. Chính xác nhưng chậm hơn vì phải xử lý từng vùng.
- **Một giai đoạn (one-stage)** như SSD [9] và YOLO [3]: một mạng duy nhất dự đoán đồng thời box và lớp cho mọi vị trí trên ảnh trong một lần lan truyền. Nhanh, phù hợp video thời gian thực.
- **Dựa trên Transformer** như DETR [10]: bỏ hẳn bước NMS, nhưng cần nhiều dữ liệu và thời gian huấn luyện hơn.

Bảng 2. *So sánh các họ thuật toán phát hiện đối tượng (định tính)*

Họ thuật toán	Ví dụ	Tốc độ	Độ chính xác	Nhu cầu dữ liệu	Phù hợp đồ án này?
Cổ điển (đặc trưng thủ công)	Viola–Jones, HOG+SVM	Nhanh	Thấp khi cảnh phức tạp	Ít	Không: xe đa dạng màu, dáng, che khuất
Hai giai đoạn	Faster R-CNN	Chậm	Cao	Vừa	Được, nhưng chậm cho video
Một giai đoạn	SSD, YOLO	Rất nhanh	Cao	Vừa	**Có**: nhanh, công cụ trọn gói
Transformer	DETR	Vừa	Cao	Nhiều	Chưa: dữ liệu nhỏ (~1.000 ảnh)

Lý do chọn YOLOv8. (i) Tốc độ đủ cho video thời gian thực, đúng với mục tiêu cuối cùng là chạy trên camera giao thông. (ii) Thư viện Ultralytics [4] gói sẵn cả vòng đời: đọc nhãn định dạng YOLO, tăng cường dữ liệu, huấn luyện, đánh giá mAP, vẽ biểu đồ và xuất ONNX, nên người mới có thể tập trung vào **dữ liệu** thay vì viết lại vòng lặp huấn luyện. (iii) Có trọng số huấn luyện sẵn trên COCO [11], một bộ dữ liệu vốn đã có lớp *motorcycle*, nên học chuyển giao rất hiệu quả. (iv) Tài liệu và cộng đồng lớn, dễ tra cứu khi gặp lỗi. Ultralytics hiện có các phiên bản mới hơn (YOLO11, YOLO26); đồ án dùng YOLOv8 vì đây là phiên bản ổn định, nhiều tài liệu nhất. Việc không so sánh với các phiên bản mới được ghi vào phần hạn chế.

Lý do chọn cỡ "s". YOLOv8 có năm cỡ n, s, m, l, x. Cỡ n (khoảng 3 triệu tham số) nhẹ nhất nhưng kém chính xác hơn; cỡ m trở lên (từ khoảng 26 triệu tham số) dễ học thuộc lòng khi chỉ có khoảng 1.000 ảnh và tốn thời gian GPU hơn. Cỡ s (11,1 triệu tham số) là điểm cân bằng: huấn luyện 100 epoch trên GPU Tesla T4 mất khoảng 20–25 phút.

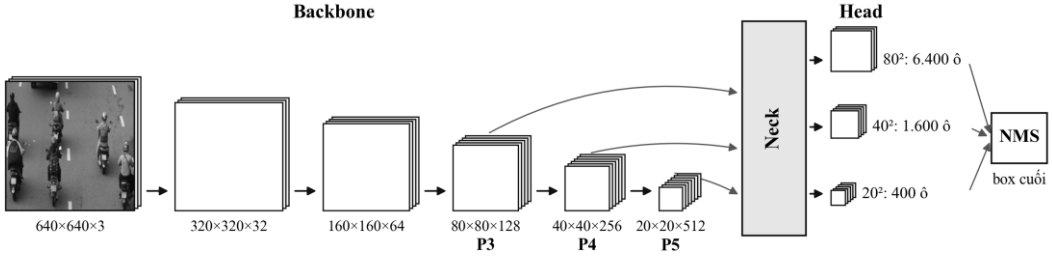
2.3. Kiến trúc YOLOv8s

Hình 2 vẽ lại kiến trúc YOLOv8s theo đúng bảng 23 lớp in ra trong nhật ký huấn luyện (§6), với ảnh đầu vào là một khung hình thật của bộ dữ liệu. Mỗi chồng tấm là một bản đồ đặc trưng: cạnh tấm tỉ lệ với độ phân giải, số tấm tăng theo số kênh. Mạng gồm ba phần:

- **Backbone** (xương sống, họ CSPDarknet [12]): các lớp tích chập bước 2 lần lượt thu nhỏ ảnh 640×640 còn 320, 160, 80, 40 và 20, trong khi số kênh tăng từ 32 lên 512. Có thể hiểu mỗi kênh là một "bộ lọc" học được, tương tự các bộ lọc Sobel hay Laplacian trong xử lý ảnh, nhưng do mạng tự học thay vì thiết kế bằng tay. Lớp nông bắt biên cạnh, lớp sâu bắt hình dạng phức tạp như "người ngồi trên xe". Khối **C2f** chia đặc trưng thành nhiều nhánh có đường tắt (residual) rồi ghép lại, giúp

mạng sâu mà vẫn học ổn định. Khối **SPPF** áp max-pooling 5×5 ba lần liên tiếp để mỗi điểm "nhìn" được vùng bối cảnh rộng.

- **Neck** (cổ, FPN [13] + PAN [14]): trộn đặc trưng giữa các tầng. Nhánh FPN đi từ tầng sâu lên tầng nông để đưa thông tin ngữ nghĩa xuống bản đồ đặc trưng có độ phân giải cao; nhánh PAN đi ngược lại để đưa thông tin vị trí chi tiết lên tầng sâu.
- **Head** (đầu ra) ở ba tỉ lệ 80×80, 40×40 và 20×20, tổng cộng 8.400 ô. Mỗi ô dự đoán trực tiếp một box mà **không cần anchor** (khung mẫu định sẵn như YOLO đời cũ). Head tách thành hai nhánh: nhánh vị trí dự đoán khoảng cách từ tâm ô tới bốn cạnh box, mỗi cạnh dưới dạng phân phối 16 mức (Distribution Focal Loss, DFL [15]); nhánh lớp dự đoán xác suất là xe máy bằng hàm sigmoid.



Hình 2. *Kiến trúc YOLOv8s với kích thước tensor thật (rộng × cao × số kênh), ảnh vào là frame 3J4A7818_0041. Mỗi mũi tên trong backbone là một Conv 3×3 bước 2, theo sau là khối C2f (riêng tầng cuối thêm SPPF). P3, P4, P5 được đưa vào neck (FPN + PAN). Head dự đoán trên 3 lưới 80×80, 40×40, 20×20, tổng 8.400 ô; NMS giữ box có điểm tin cậy ≥ 0,268 và loại box chồng nhau có IoU > 0,7*

Hàm mất mát. Khi huấn luyện, mỗi box thật được gán cho một số ô "phù hợp nhất" bằng bộ gán nhãn TAL [16], dựa trên cả điểm phân loại lẫn độ khớp box. Tổng mất mát gồm ba phần, đúng ba đường cong sẽ thấy ở Hình 10: **box loss** dùng CIoU [17] (phạt khi box lệch về diện tích chồng, khoảng cách tâm và tỉ lệ cạnh), **DFL loss** cho phân phối 16 mức của từng cạnh, và **cls loss** là entropy chéo nhị phân (BCE) cho xác suất là xe máy. Trọng số mặc định lần lượt là 7,5; 1,5 và 0,5.

Hậu xử lý. 8.400 ô cho ra 8.400 box ứng viên, nhiều box cùng chồng lên một chiếc xe. Bước **NMS** (non-maximum suppression) giữ box có điểm tin cậy cao nhất rồi loại các box khác chồng lên nó quá 70% (IoU > 0,7). Sau đó chỉ giữ box có điểm tin cậy vượt ngưỡng; ngưỡng này được chọn ở mục 4.3.

2.4. Học chuyển giao

Thay vì khởi tạo ngẫu nhiên, mô hình bắt đầu từ trọng số đã huấn luyện trên COCO [11] (khoảng 118 nghìn ảnh, 80 lớp). Nhật ký ghi "Transferred 349/355 items", nghĩa là gần như toàn bộ backbone và neck được giữ lại; chỉ các lớp cuối của head phải học lại vì số lớp đổi từ 80 xuống 1. Lý do: các đặc trưng ở tầng nông và tầng giữa (biên, góc, kết cấu, hình dạng bánh xe, người) có tính tổng quát và chuyển giao tốt giữa các bài toán [18]. Với chỉ khoảng 1.000 ảnh, học chuyển giao giúp hội tụ nhanh hơn và giảm nguy cơ học thuộc lòng rất nhiều so với huấn luyện từ đầu.

2.5. Thước đo đánh giá

IoU (Intersection over Union) đo mức chồng khít giữa hai box P và G, thường là box dự đoán và box thật:

$$\text{IoU} = |P \cap G| / |P \cup G| \qquad (1)$$

trong đó $|\cdot|$ là diện tích. IoU bằng 1 khi hai box trùng khít và bằng 0 khi không chạm nhau. Hình 3a cho một ví dụ thật: frame 3J4A7820_0016 bị upload hai lần, một bản nằm ở train và một bản ở valid, mỗi bản được gán nhãn riêng. Hai box của cùng một chiếc xe chỉ đạt IoU = 0,883. Ví dụ này cho thấy cùng lúc cả rò rỉ dữ liệu lẫn nhiễu nhãn (mục 3.4).

Một dự đoán là **TP** (đúng) nếu IoU với một box thật chưa được ghép đạt ít nhất 0,5;

nếu không ghép được với box thật nào thì là **FP** (báo nhầm); box thật không được dự đoán nào ghép là **FN** (bỏ sót). Từ đó tính:

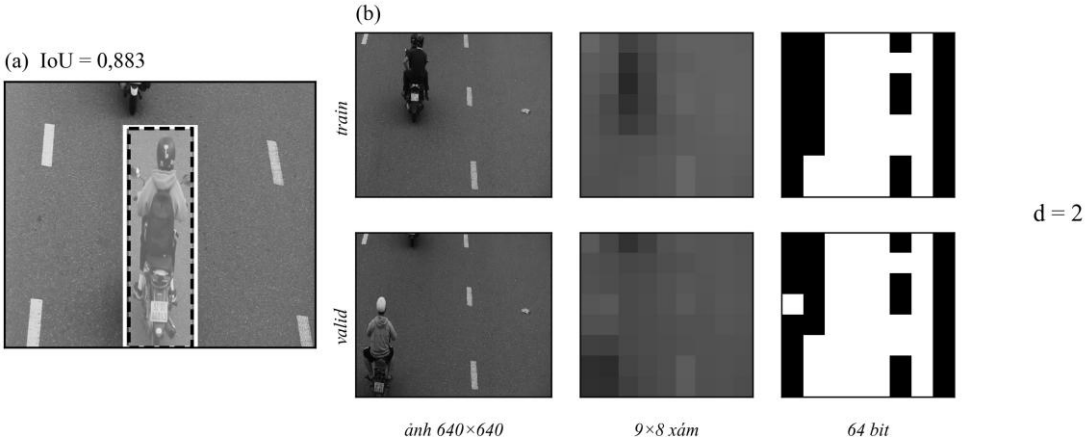
$$P = TP / (TP + FP) \quad (2)$$

$$R = TP / (TP + FN) \quad (3)$$

$$F1 = 2 \cdot P \cdot R / (P + R) \quad (4)$$

Precision *P* trả lời "trong các box mô hình vẽ, bao nhiêu phần trăm đúng", Recall *R* trả lời "trong các xe thật, mô hình tìm được bao nhiêu phần trăm". Hai đại lượng này phụ thuộc ngưỡng tin cậy: hạ ngưỡng thì *R* tăng, *P* giảm. Để có một con số không phụ thuộc ngưỡng, ta quét mọi ngưỡng, vẽ đường *P* theo *R* và lấy diện tích dưới đường cong, gọi là **AP** (average precision) [19]. Vì chỉ có một lớp nên mAP = AP. Có hai biến thể:

- **mAP@0.5**: TP khi IoU ≥ 0,5, tức là chấm "dễ", chỉ cần box đúng chỗ.
- **mAP@0.5:0.95**: trung bình AP ở 10 ngưỡng IoU 0,50; 0,55; …; 0,95 theo chuẩn COCO [11], tức là chấm "khó", box phải khít gần như tuyệt đối mới được điểm ở các ngưỡng cao.



Hình 3. *(a) Hai lần gán nhãn của cùng một ảnh (nét liền trắng: bản ở train; nét đứt đen: bản ở valid); vùng sáng là phần giao, IoU = 0,883. (b) dHash trên hai frame thật của video 7829 (train frame_0193, valid frame_0036): ảnh gốc, ảnh thu về 9×8 mức xám và 64 bit (trắng: điểm bên phải sáng hơn điểm bên trái). Hai vân tay chỉ khác 2 bit (d = 2) dù người lái khác nhau, vì dHash chủ yếu phản ánh nền đường (mục 2.6)*

2.6. Các kỹ thuật xử lý ảnh dùng để khảo sát dữ liệu

Phần kiểm tra chất lượng (§4) chỉ dùng kiến thức xử lý ảnh số cơ bản, áp lên ảnh mức xám *g(x, y)*:

- **Độ sáng** = giá trị trung bình của *g*; **độ tương phản** = độ lệch chuẩn của *g* (mức xám càng trải rộng thì tương phản càng cao).
- **Độ nét** = phương sai của ảnh sau khi lọc Laplacian [20]. Toán tử Laplacian là đạo hàm bậc hai, cho giá trị lớn ở biên cạnh sắc. Ảnh mờ có ít biên sắc nên phương sai thấp. Cách đo này rẻ, không cần ảnh tham chiếu, nên phù hợp để rà nhanh hàng nghìn ảnh.
- **dHash** (difference hash) [21]: thu ảnh về 9×8 điểm mức xám, so mỗi điểm với điểm bên phải, được 8×8 = 64 bit làm "vân tay" của ảnh (Hình 3b). Hai ảnh giống nhau có vân tay giống nhau; số bit khác nhau gọi là **khoảng cách Hamming**.

Vì sao dùng dHash mà không dùng cách khác? Băm mật mã như MD5 chỉ phát hiện file giống hệt từng byte, mà ảnh upload lại qua Roboflow bị nén lại nên các byte khác nhau dù nội dung như nhau. Đặc trưng CNN (embedding) mạnh hơn nhưng phải chạy một mạng nơ-ron trên mọi ảnh. dHash chịu được nén và đổi độ sáng nhẹ, tính rất nhanh và dễ giải thích. Đổi lại, dHash chỉ nhìn cấu trúc sáng–tối thô của toàn ảnh. Với camera cố định, mặt đường chiếm phần lớn khung hình,

nên hai frame chở người khác nhau vẫn có thể có vân tay gần giống (Hình 3b). Vì vậy, notebook **không** dùng dHash một mình để kết luận "trùng": mọi cặp ứng viên đều được xác nhận lại bằng cách so trực tiếp ảnh thu nhỏ 64×64. Ngưỡng ở bước này là sai khác trung bình dưới 3 mức xám.

2.7. Rò rỉ dữ liệu và chia dữ liệu theo nhóm

Rò rỉ dữ liệu (data leakage) [22] xảy ra khi thông tin của tập kiểm định lọt vào tập huấn luyện, làm điểm đánh giá đẹp hơn thực tế, giống học sinh được xem trước đề thi. Với dữ liệu cắt từ video, hai frame liền nhau gần như giống hệt. Nếu chia ngẫu nhiên từng ảnh, frame thứ 10 nằm ở train còn frame thứ 11 nằm ở valid, và mô hình chỉ cần "nhớ" là đủ đúng.

Giải pháp là **chia theo nhóm**: mọi frame của cùng một video phải nằm chung một phía. Notebook dùng `GroupShuffleSplit` của scikit-learn [23] với nhóm là mã video. So với chia ngẫu nhiên, cách này cho ước lượng trung thực về khả năng làm việc trên **video mới**. So với K-fold theo nhóm (GroupKFold), cách này chỉ cần huấn luyện một lần nên rẻ hơn, nhưng kết quả phụ thuộc vào video nào được chọn làm kiểm định (xem mục 5).

3. Dữ liệu và phương pháp thực hiện

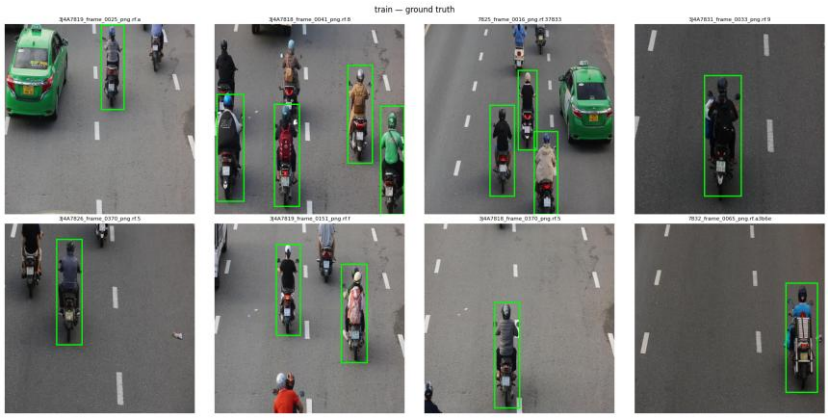
3.1. Mô tả dữ liệu

Dữ liệu là bộ `motobike-detection` phiên bản 1 trên Roboflow Universe (giấy phép CC BY 4.0), tải qua Kaggle [5]. Ảnh là các khung hình cắt từ 13 video quay từ trên cao xuống một con đường nhiều làn, xe đi theo hướng ra xa camera. Tên file có dạng `3J4A7825_frame_0016…` hoặc `7825_frame_0016…`; notebook coi bốn chữ số trước "_frame" là mã video (hàm `video_id`).

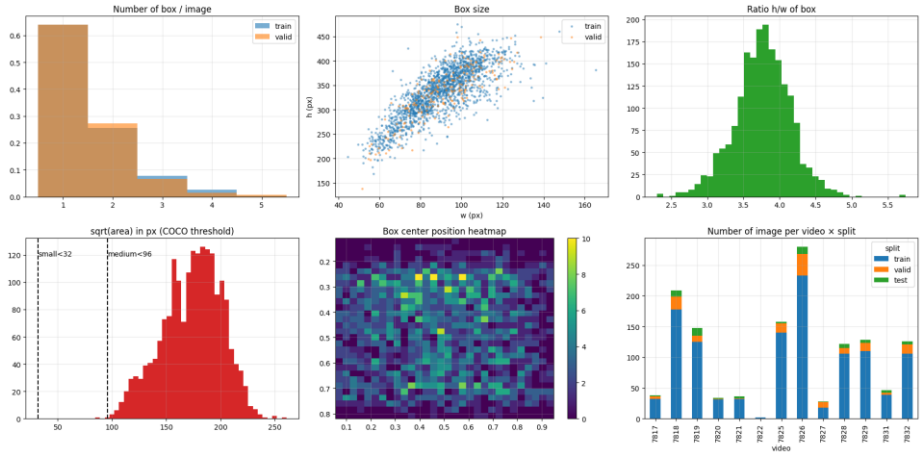
Bảng 3. *Đặc điểm bộ dữ liệu*

Thuộc tính	Giá trị
Số lớp	1 (`motobike`)
Kích thước ảnh	640×640 (đã resize kiểu stretch)
Train / valid / test	1.152 / 136 / 67 ảnh
Nhãn tập test	Không có
Số video	13 (từ 2 đến 280 ảnh/video)
Số box train / valid	1.723 / 201
Định dạng nhãn	`class cx cy w h`, chuẩn hoá 0–1

Hai điểm cần lưu ý ngay từ đầu. Thứ nhất, tập test **không có nhãn**, nên mọi con số định lượng trong báo cáo đều tính trên tập kiểm định. Thứ hai, Roboflow đã resize mọi ảnh về 640×640 bằng cách **kéo giãn** (không giữ tỉ lệ khung hình), nên hình dạng xe trong dữ liệu có thể đã bị méo so với video gốc.



Hình 4. *Tám ảnh train ngẫu nhiên với nhãn gốc (khung xanh lá). Có thể thấy nhiều xe rõ ràng không được gán nhãn, ví dụ hai xe ở giữa phía trên ảnh thứ hai và xe bên trái ảnh thứ năm; điều này sẽ được phân tích ở mục 4.4*



Hình 5. *Khảo sát nhãn: số box mỗi ảnh, kích thước box (px), tỉ lệ cao/rộng, căn bậc hai diện tích so với ngưỡng COCO, bản đồ nhiệt vị trí tâm box và số ảnh theo từng video*

3.2. Khảo sát nhãn (EDA)

Hình 5 (§3) tóm tắt phân bố nhãn. Các nhận xét chính:

- **Cảnh thưa:** trung bình 1,50 box/ảnh ở train và 1,48 ở valid; khoảng 64% ảnh chỉ có một box. Có 1 ảnh train không có box nào.
- **Box cao và hẹp:** trung vị rộng 0,142 và cao 0,533 (khoảng 91×341 px), tỉ lệ cao/rộng trung bình 3,77. Hình dáng "người ngồi trên xe nhìn từ phía sau" vốn cao, nhưng việc kéo giãn ảnh có thể làm tỉ lệ này lớn hơn thực tế.
- **Toàn bộ là vật lớn:** căn bậc hai diện tích box nằm trong khoảng 90–260 px, đều vượt ngưỡng 96 px mà COCO coi là "vật lớn". Vì vậy không cần các kỹ thuật dành riêng cho vật nhỏ như tăng độ phân giải đầu vào hay cắt ảnh thành ô.
- **Vị trí:** tâm box trải đều theo chiều ngang (các làn đường) và nằm trong khoảng 20–80% chiều cao ảnh.
- **Mất cân bằng giữa các video:** video 7826 có khoảng 280 ảnh, video 7822 chỉ có 2 ảnh. Đặc biệt, **mọi video đều có mặt ở cả train lẫn valid** trong cách chia gốc, là dấu hiệu đầu tiên của rò rỉ.

3.3. Kiểm tra chất lượng ảnh và nhãn

Thống kê ảnh (§4.1). Độ sáng trung bình khoảng 100/255 (train 101,6; valid 99,8; test 103,6), nghĩa là ảnh hơi tối. Độ tương phản khoảng 26, thấp, phù hợp với cảnh mặt đường xám đồng đều. Độ nét (phương sai Laplacian) khoảng 640 ở train/valid và 709 ở test. Phân bố của ba tập gần như chồng lên nhau, nên **không có lệch phân bố** về ánh sáng hay độ nét giữa các tập. Tuy nhiên, trong mỗi tập độ sáng trải khá rộng, từ 57 đến 149, nên vẫn cần tăng cường màu (mục 3.6) để mô hình chịu được ánh sáng khác khi gặp camera mới.

Kiểm tra nhãn (§4.2). Đoạn mã kiểm tra sáu loại lỗi: toạ độ tâm nằm ngoài [0, 1], box tràn ra ngoài ảnh, box suy biến (cạnh nhỏ hơn 0,5% kích thước ảnh, khoảng 3 px), hai box trùng nhau trong cùng ảnh (IoU > 0,9), mã lớp sai và ảnh không có nhãn. Kết quả chỉ có **một vấn đề**: một ảnh train có file nhãn rỗng. YOLO coi ảnh như vậy là **ảnh nền** (ví dụ âm), có ích để mô hình học "chỗ này không có xe", nên được giữ lại.

3.4. Phát hiện trùng lặp và rò rỉ

Đây là bước khai phá quan trọng nhất của đồ án (§4.3). Quy trình gồm bốn bước:

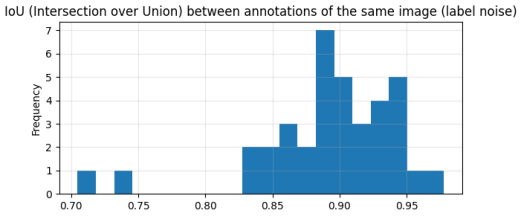
- **Tạo cặp ứng viên:** hai ảnh có cùng tên frame (khác mã băm của Roboflow) hoặc có dHash giống hệt nhau.
- **Xác nhận bằng pixel:** thu cả hai ảnh về 64×64 mức xám; nếu sai khác tuyệt đối trung bình nhỏ hơn 3 mức xám thì kết luận là cùng một ảnh.

- **Gom nhóm bằng union-find:** nếu A trùng B và B trùng C thì A, B, C cùng một nhóm. Cấu trúc này gộp nhóm gần như tức thì.
- **Đo rò rỉ do frame liền kề:** với mỗi ảnh valid và test, tìm khoảng cách Hamming nhỏ nhất tới tập train; nếu ≤ 5 thì coi là gần trùng.

Bảng 4. *Kết quả kiểm tra trùng lặp và rò rỉ*

Phát hiện	Số liệu
Nhóm trùng tên frame	182 nhóm, chỉ 43 nhóm trùng pixel
Nhóm ảnh trùng hệt nhau	45 nhóm / 90 ảnh (45 ảnh thừa)
…chỉ nằm trong train	33 nhóm
…vắt qua test–train	7 nhóm
…vắt qua train–valid	4 nhóm
…vắt qua test–valid	1 nhóm
Cặp trùng có cùng số box	100%
IoU TB giữa 2 lần gán nhãn	0,892
Valid gần trùng train (≤ 5)	24,3%
Test gần trùng train (≤ 5)	40,3%

Bảng 4 cho ba bài học. **Một**, tên file không đáng tin: trong 182 nhóm trùng tên chỉ có 43 nhóm trùng nội dung, vì các video khác nhau dùng chung cách đánh số frame. **Hai**, có rò rỉ trực tiếp: 12 nhóm ảnh trùng hệt nằm vắt qua hai tập (Hình 7 cho ba ví dụ valid–train có khoảng cách 0). **Ba**, nhãn có nhiễu: cùng một ảnh được gán nhãn hai lần cho số box giống nhau, nhưng box chỉ khớp nhau với IoU trung bình 0,892 (Hình 6). Nói cách khác, người gán nhãn vẽ lệch so với chính mình khoảng 11%, và đây sẽ là **trần tự nhiên** cho chỉ số mAP@0.5:0.95 (mục 4.2).



Hình 6. *Phân bố IoU giữa hai lần gán nhãn của cùng một ảnh (đo nhiễu nhãn)*

Với rò rỉ do frame liền kề, các tỉ lệ 24,3% và 40,3% nên được hiểu là **ước lượng thô** theo hướng cao. Như Hình 3b cho thấy, dHash có thể coi hai frame cùng camera là gần trùng dù xe trong ảnh khác nhau. Dù vậy, kết luận chung vẫn đứng vững: cách chia gốc làm valid quá giống train, nên điểm trên valid gốc sẽ lạc quan quá mức.



Hình 7. *Ba cặp valid (hàng trên) – train (hàng dưới) giống nhau nhất: khoảng cách Hamming bằng 0, là cùng một ảnh được upload hai lần*

3.5. Tiền xử lý: làm sạch, loại trùng và chia theo video

Hàm `clean_labels` (§5.1) áp cho cả hai phiên bản: cắt box về trong ảnh, bỏ box suy biến, bỏ mã lớp sai và bỏ box trùng. Trên bộ dữ liệu này không box nào bị loại (1.723 + 201 = 1.924 box giữ nguyên), nhưng bước này vẫn cần như một lớp bảo vệ khi dữ liệu thay đổi. Sau đó hai phiên bản được tạo như sau:

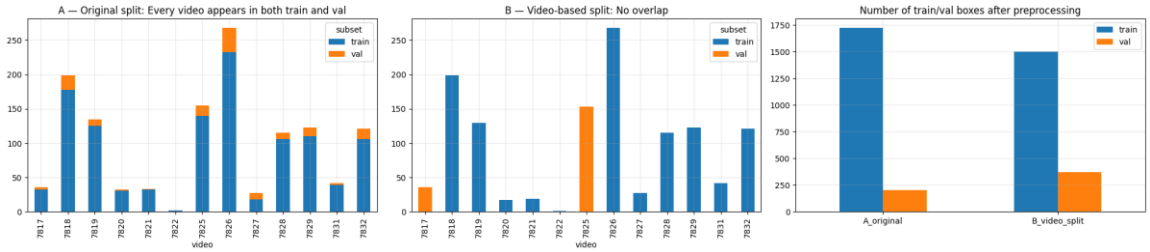
- **Bản A (split gốc):** giữ nguyên cách chia của Roboflow; đóng vai trò mốc so sánh "có rò rỉ".
- **Bản B (sạch + theo video):** trong mỗi nhóm trùng chỉ giữ một ảnh (ưu tiên ảnh có nhiều box nhất), còn 1.251 trên 1.288 ảnh có nhãn. Sau đó chạy `GroupShuffleSplit` với 500 hạt giống ngẫu nhiên và chọn cách chia có tỉ lệ valid gần 15% nhất mà valid có ít nhất 2 video. Kết quả: valid gồm video **7817** và **7825**, chiếm 15,1% số ảnh.

Chỉ có 37 ảnh bị loại chứ không phải 45, vì một số bản sao nằm ở tập test, vốn không thuộc dữ liệu có nhãn.

Bảng 5. *So sánh hai phiên bản dữ liệu sau tiền xử lý*

	A – gốc	**B – theo video**
Ảnh train / val	1.152 / 136	1.062 / 189
Box train / val	1.723 / 201	1.501 / 369
Box/ảnh train	1,50	1,41
Box/ảnh val	1,48	**1,95**
Video ở val	cả 13 video	7817, 7825
Rò rỉ	Có	Không (theo video)

Bảng 5 cho thấy một điểm tinh tế: val của B **đông xe hơn** train của B (1,95 so với 1,41 box/ảnh), do video 7825 là đoạn đường đông. Vì vậy, khi điểm của B thấp hơn A, phần chênh lệch đến từ **cả** việc hết rò rỉ **lẫn** việc val của B khó hơn.



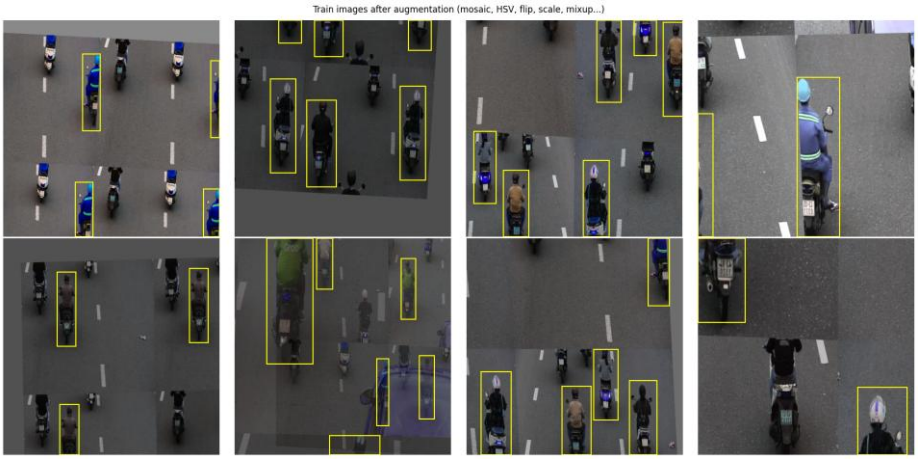
Hình 8. *Phân bố ảnh theo video: bản A (trái) có mọi video ở cả train và val; bản B (giữa) tách riêng video 7817 và 7825 làm val; số box train/val của hai bản (phải)*

3.6. Tăng cường dữ liệu

Với khoảng 1.000 ảnh từ vài camera cố định, mô hình dễ "học thuộc" bối cảnh. Tăng cường dữ liệu (augmentation) tạo ra biến thể mới của ảnh ở **mỗi lần đọc** (online), nên trong 100 epoch mô hình gần như không thấy lại đúng một ảnh hai lần. Mỗi tham số trong Bảng 6 đều được chọn dựa trên kết quả khảo sát ở trên. Ngoài ra, Ultralytics còn tự thêm làm mờ, chuyển xám và CLAHE với xác suất rất nhỏ (1%).

Bảng 6. *Các phép tăng cường dữ liệu, giá trị và lý do lựa chọn*

Phép biến đổi	Tham số	Ý nghĩa	Lý do (dựa trên EDA)
Mosaic [24]	`mosaic=1.0`	Ghép 4 ảnh thành 1	Ít bối cảnh → tạo cảnh mới, nhiều xe/ảnh hơn
Tắt mosaic cuối	`close_mosaic=10`	10 epoch cuối dùng ảnh thật	Cho mô hình làm quen lại ảnh nguyên vẹn
Phóng to/thu nhỏ	`scale=0.5`	Tỉ lệ ngẫu nhiên 0,5–1,5	Xe gần/xa có kích thước khác nhau
Dịch chuyển	`translate=0.1`	Dời ảnh tối đa 10%	Xe xuất hiện ở mọi làn
Xoay, xiên	`degrees=5`, `shear=2`	Xoay ±5°, xiên ±2°	Camera có thể hơi nghiêng; không xoay mạnh vì xe luôn đứng
Lật ngang	`fliplr=0.5`	Lật trái–phải 50%	Xe đi làn trái hay phải đều hợp lý
Lật dọc	`flipud=0.0`	Không dùng	Xe lộn ngược không bao giờ xảy ra
Màu HSV	`hsv_h=0.015`, `hsv_s=0.6`, `hsv_v=0.45`	Đổi sắc độ, độ bão hoà, độ sáng	Độ sáng trải 57–149; camera mới có thể khác
MixUp [25]	`mixup=0.1`	Chồng mờ 2 ảnh, 10% số lần	Giảm học thuộc lòng



Hình 9. *Ảnh train sau tăng cường (mosaic, HSV, lật, co giãn…) với box vàng; vùng xám là phần đệm sau khi dịch/xoay*

3.7. Thiết lập huấn luyện

Hai thí nghiệm A và B dùng **cùng một hàm** `train_run` (§6) với thiết lập trong Bảng 7. Môi trường: Kaggle, GPU Tesla T4 (16 GB), Python 3.13, PyTorch 2.11, Ultralytics 8.4.174.

Bảng 7. *Siêu tham số huấn luyện và lý do*

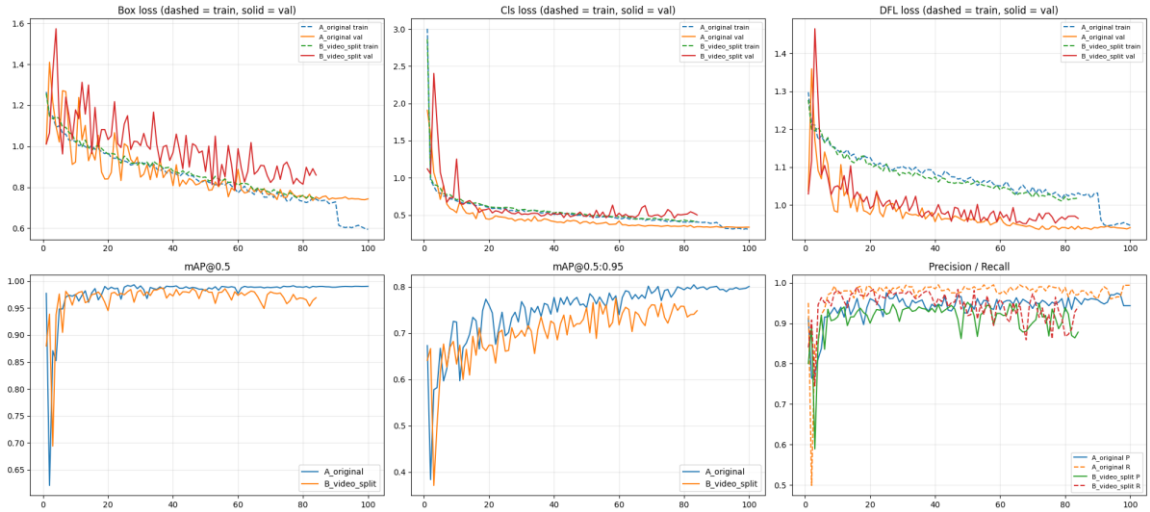
Tham số	Giá trị	Lý do
Mô hình khởi đầu	`yolov8s.pt` (COCO)	Học chuyển giao, xem mục 2.4
Kích thước ảnh	640	Bằng kích thước gốc → không mất chi tiết, không tốn tính toán thừa
Số epoch tối đa	100	Đủ để mAP bão hoà (Hình 10)
Batch	16	Vừa bộ nhớ T4 (dùng khoảng 4,3 GB); 72 bước/epoch với A, 67 với B
Bộ tối ưu	AdamW, lr = 0,002, momentum 0,9	`optimizer=auto`: Ultralytics chọn AdamW [26] khi tổng số bước < 10.000 (ở đây 7.200)
Lịch học	cosine, 3 epoch khởi động	`cos_lr=True` [27]: giảm tốc độ học êm, tinh chỉnh tốt ở cuối
Weight decay	0,0005	Điều chuẩn L2, hạn chế trọng số quá lớn
Dừng sớm	`patience=25`	Dừng nếu mAP@0.5:0.95 trên val không tăng sau 25 epoch
Độ chính xác hỗn hợp	AMP (FP16)	Nhanh hơn, ít bộ nhớ hơn trên T4
Tái lập	`seed=42`, `deterministic=True`	Chạy lại cho cùng kết quả

4. Kết quả và bàn luận

4.1. Quá trình huấn luyện

Bản A chạy hết 100 epoch trong 24 phút, epoch tốt nhất là 83. Bản B **dừng sớm** ở epoch 84 sau 20 phút vì mAP@0.5:0.95 không cải thiện kể từ epoch 59. Hình 10 so sánh đường cong của hai bản; từ đó rút ra ba nhận xét:

- **Dấu hiệu rò rỉ ở A:** loss trên val của A gần như dính sát loss trên train, vì "đề thi" quá giống "đề ôn". Ở B, val box loss cao hơn và dao động mạnh hơn (khoảng 0,8–0,9 so với 0,75 của train): đây mới là bài kiểm tra thật trên video mới.
- **Không thấy học thuộc lòng rõ rệt:** dấu hiệu kinh điển là loss trên val tăng trở lại trong khi loss trên train tiếp tục giảm. Ở đây loss trên val chỉ đi ngang, mAP bão hoà từ khoảng epoch 40–60.
- **Bước nhảy ở epoch 91 của A:** train box loss rơi đột ngột từ khoảng 0,73 xuống 0,61. Đây không phải học thuộc lòng mà là do `close_mosaic` tắt ghép ảnh, khiến ảnh train "dễ" hơn. Bản B dừng ở epoch 84 nên **chưa bao giờ** được hưởng giai đoạn này, đây là một điểm khiến so sánh A–B chưa hoàn toàn công bằng.



Hình 10. *Đường cong huấn luyện của A (xanh dương/cam) và B (xanh lá/đỏ): box, cls, DFL loss (nét đứt là train, nét liền là val), mAP@0.5, mAP@0.5:0.95, precision và recall theo epoch*

4.2. So sánh hai phiên bản dữ liệu

Bảng 8. *Kết quả trên tập kiểm định của từng bản với trọng số tốt nhất (best.pt)*

Bản dữ liệu	Precision	Recall	mAP@0.5	mAP@0.75	mAP@0.5:0.95
A – split gốc (rò rỉ)	0,964	0,970	0,990	0,968	0,806
B – theo video (trung thực)	0,924	0,946	0,980	0,950	0,775
Chênh lệch A − B	0,040	0,024	0,010	0,017	0,031

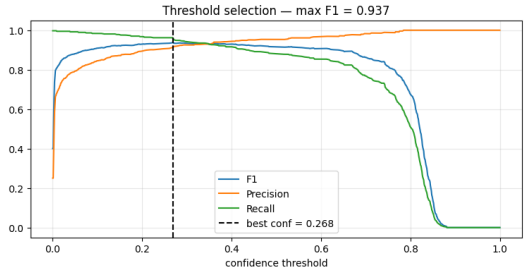
Bảng 8 trả lời câu hỏi CH2. **Thứ nhất**, mô hình hoạt động tốt trên video chưa từng thấy: với mAP@0.5 = 0,980, gần như mọi xe đều được tìm thấy ở đúng vị trí. **Thứ hai**, rò rỉ chỉ làm điểm đẹp lên **một chút**: 0,01 với mAP@0.5 và 0,03 với mAP@0.5:0.95. Mô hình không chỉ học thuộc mà thực sự học được khái niệm "xe máy nhìn từ trên cao".

Cần đọc con số chênh lệch một cách thận trọng vì ba lý do. (i) Hai bản được chấm trên **hai tập val khác nhau**, mà val của B đông xe hơn (mục 3.5). (ii) B không có giai đoạn `close_mosaic` (mục 4.1). (iii) Trên Hình 10, mAP@0.5:0.95 của B dao động khoảng ±0,03 giữa các epoch liền nhau, tức là bằng chính độ lớn của chênh lệch. Kết luận an toàn là: "rò rỉ có làm điểm đẹp lên, nhưng không nhiều", chứ chưa thể nói chính xác bao nhiêu.

Vì sao mAP@0.5:0.95 chỉ khoảng 0,78? Ở các ngưỡng IoU 0,9 và 0,95, box dự đoán phải khít gần như tuyệt đối. Nhưng như mục 3.4 đã đo, chính người gán nhãn vẽ cùng một ảnh hai lần cũng chỉ khớp nhau ở mức IoU 0,89. Biểu đồ IoU của các TP ở Hình 12 cho thấy đa số dự đoán đúng khớp với nhãn ở mức 0,85–0,95, tức là mô hình đã **khít ngang người gán nhãn**. Muốn tăng chỉ số này, cần nhãn chính xác hơn trước khi nghĩ tới mô hình lớn hơn.

4.3. Chọn ngưỡng tin cậy

Điểm tin cậy của mỗi box nằm trong khoảng 0–1; ta chỉ giữ box có điểm vượt ngưỡng. Notebook (§7.3) chọn ngưỡng làm F1 lớn nhất trên val của B: **0,268**, với F1 = 0,937 (Hình 11). Đường F1 gần như **phẳng** trong khoảng 0,15–0,6, nên chọn 0,25, 0,3 hay 0,4 đều cho kết quả gần như nhau, tức là mô hình ít nhạy với ngưỡng. Nếu ứng dụng ưu tiên ít báo nhầm (ví dụ đếm xe) thì có thể dùng ngưỡng khoảng 0,5 mà recall chỉ giảm nhẹ.



Hình 11. *F1, precision và recall theo ngưỡng tin cậy trên val của B; nét đứt là ngưỡng được chọn*

4.4. Phân tích lỗi

Thay vì chỉ nhìn mAP, notebook (§7.4) tự ghép từng dự đoán với nhãn thật bằng IoU ≥ 0,5 theo thứ tự điểm tin cậy giảm dần, rồi đếm TP/FP/FN ở ngưỡng 0,268 trên 189 ảnh val của B (Bảng 9).

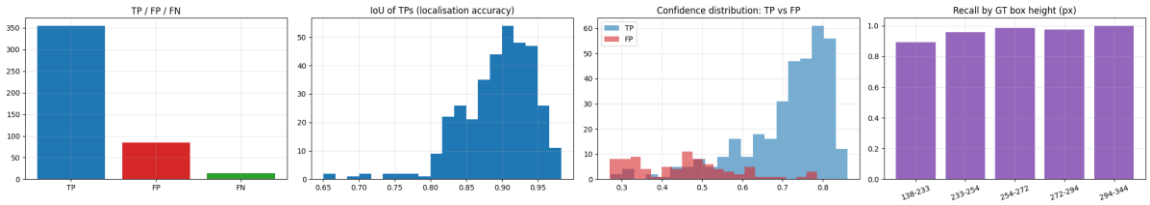
Bảng 9. *Kết quả ghép dự đoán với nhãn thật (val B, ngưỡng 0,268)*

TP	FP	FN	Precision	Recall
355	85	14	0,807	0,962

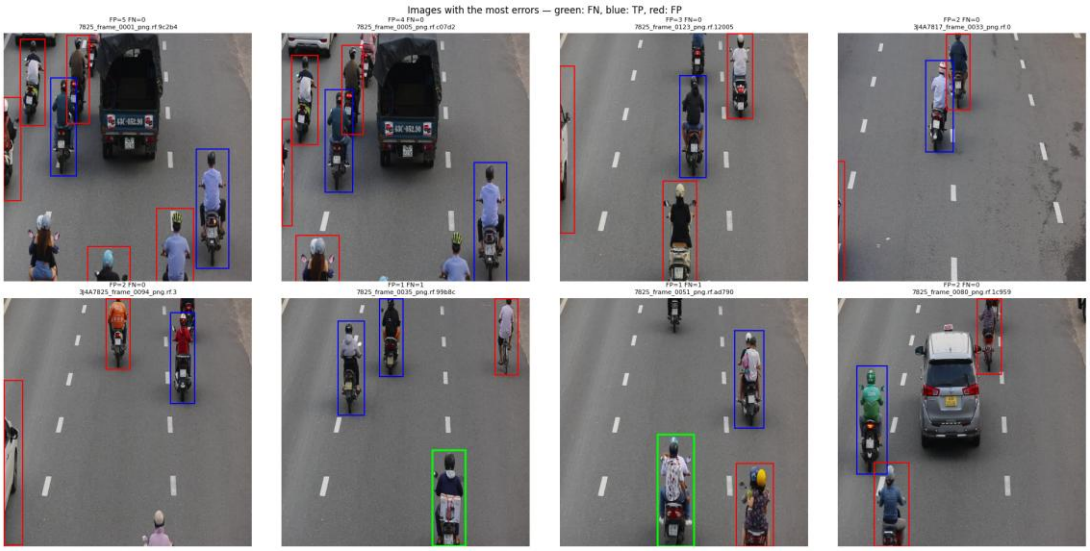
Recall rất cao (0,962) nhưng precision chỉ 0,807: có 85 lần "báo nhầm". Tuy nhiên, khi xem trực tiếp các ảnh có nhiều lỗi nhất (Hình 13) thì phần lớn khung đỏ (FP) lại nằm trên **xe máy có thật mà người gán nhãn bỏ sót**. Ví dụ, ở frame 7825_0001 bị tính 5 FP, cả năm khung đỏ đều là người đang đi xe máy: một xe ở góc trên bên trái, hai xe khuất sau xe tải và hai xe bị cắt ở mép dưới. Hiện tượng tương tự lặp lại ở các frame 0005, 0094, 0080 và 0051. Hình 4 cho thấy tình trạng thiếu nhãn này có cả trong tập train. Đây là trường hợp "**thước đo sai chứ không phải học sinh sai**", một hiện tượng phổ biến kể cả ở các bộ dữ liệu chuẩn lớn [28].

Lỗi thật của mô hình thuộc hai nhóm. **FP thật:** box dọc vẽ lên phần ô tô bị cắt ở mép trái ảnh (thấy ở ba ảnh), và có thể một trường hợp nhầm xe đạp ở mép phải. Mô hình nhầm vật thể dạng hình chữ nhật đứng ở mép ảnh là xe máy. **FN:** biểu đồ recall theo chiều cao box (Hình 12, phải) cho thấy nhóm 20% xe nhỏ nhất (cao 138–233 px) chỉ được tìm thấy khoảng 89%, trong khi các nhóm còn lại đạt 96–100%. Xe nhỏ là xe ở xa hoặc bị che một phần.

Biểu đồ phân bố điểm tin cậy (Hình 12) bổ sung một bằng chứng nữa: TP tập trung ở 0,7–0,85, còn FP chủ yếu ở 0,27–0,6. Điều này hợp lý: trong dữ liệu train, những chiếc xe giống nhau lúc được gán nhãn, lúc không, nên mô hình "phân vân" với chúng.



Hình 12. *Phân tích lỗi trên val B: số TP/FP/FN, IoU của các TP, phân bố điểm tin cậy của TP và FP, recall theo chiều cao box thật*



Hình 13. *Tám ảnh val có nhiều lỗi nhất: khung xanh dương là TP, đỏ là FP, xanh lá là FN. Phần lớn khung đỏ nằm trên xe máy thật nhưng không có trong nhãn*

Một điểm chưa giải thích được. Hàm đánh giá của Ultralytics báo precision 0,924 và recall 0,946 tại ngưỡng tối ưu F1 của nó (khoảng 0,29), còn phép đếm tự viết ở trên cho 0,807 và 0,962 tại ngưỡng 0,268, dù dùng cùng dữ liệu. Khác biệt về ngưỡng quá nhỏ để giải thích chênh lệch này, vì đường F1 gần như phẳng quanh đó. Nguyên nhân có thể nằm ở khác biệt giữa hai đường xử lý `predict()` và `val()` hoặc ở cách ghép cặp. Báo cáo ghi rõ nguồn của từng con số và coi đây là việc cần kiểm tra tiếp (mục 5).

4.5. Suy luận trên tập test và hiệu năng

Mô hình B chạy trên 67 ảnh test (§8) và vẽ 119 box, không ảnh nào trống. Phần lớn box có điểm tin cậy từ 0,8 trở lên. Số box dự đoán trung bình khoảng 1,78/ảnh, cao hơn mức 1,50/ảnh của nhãn train, khớp với nhận định ở mục 4.4 rằng mô hình tìm được cả những xe mà người gán nhãn hay bỏ qua. Có 8 ảnh test giống hệt ảnh train, nên kết quả trên các ảnh này đẹp hơn thực tế. Thời gian suy luận khoảng 5–10 ms/ảnh trên T4, cộng thêm khoảng 3 ms tiền và hậu xử lý, tức là trên 60 khung hình/giây, đủ cho video thời gian thực. Mô hình được lưu dưới dạng `.pt` (22,5 MB) và `.onnx` (42,7 MB, đầu ra 1×5×8.400) để triển khai ngoài Python.

4.6. Tri thức rút ra

Tổng hợp lại, đồ án khai phá được các tri thức sau, xếp theo mức độ quan trọng:

- **Về dữ liệu:** bộ dữ liệu có rò rỉ thật (45 nhóm ảnh trùng, 12 nhóm vắt qua các tập) và nhãn **không đầy đủ**: nhiều xe máy nhìn thấy rõ không được gán nhãn, cả ở train lẫn val. Đây là yếu tố giới hạn chất lượng lớn nhất.
- **Về độ khó của bài toán:** với camera cố định nhìn từ trên cao, xe máy là vật thể lớn, rõ nét. Một mô hình cỡ nhỏ đã đạt mAP@0.5 khoảng 0,98 trên video mới; phần còn lại chủ yếu là xe nhỏ, ở xa và vật bị cắt ở mép ảnh.
- **Về đánh giá:** mAP@0.5:0.95 bị chặn bởi nhiễu nhãn (IoU giữa 2 lần gán nhãn khoảng 0,89). Precision tính theo nhãn hiện tại đánh giá **thấp** chất lượng thật của mô hình.
- **Về phương pháp:** chia theo video là bắt buộc với dữ liệu cắt từ video. Ở bộ dữ liệu này, chênh lệch do rò rỉ nhỏ, nhưng nếu không chia lại thì không có cách nào biết điều đó.
- **Về giao thông:** phần lớn khung hình chỉ có 1–2 xe máy được gán nhãn, còn video 7825 đông hơn hẳn (gần 2 xe/ảnh). Do

nhãn thiếu, các con số này là **cận dưới** của mật độ thật.

5. Hạn chế và hướng phát triển

5.1. Hạn chế

Về dữ liệu.

- Chỉ một kiểu khung cảnh: camera trên cao, ban ngày, trời khô, xe đi ra xa. Chưa có dữ liệu ban đêm, trời mưa hay góc nhìn khác, nên chưa biết mô hình hoạt động thế nào ngoài điều kiện này.
- Nhãn thiếu và có nhiễu (mục 4.4 và 3.4), làm sai lệch cả quá trình huấn luyện lẫn mọi con số đánh giá.
- Ảnh đã bị kéo giãn về 640×640; khi chạy trên video gốc phải kéo giãn y hệt, nếu không hình dạng xe sẽ khác lúc huấn luyện.
- Chỉ có một lớp `motobike`: không có lớp xe đạp hay ô tô làm "đối trọng", nên mô hình dễ nhầm các vật hình chữ nhật đứng.
- Tập test không có nhãn, và có 8 ảnh trùng với train.
- Mã video được suy ra từ tên file (giả định `3J4A7825` và `7825` là cùng một video) và chưa được kiểm chứng độc lập.
- Tỉ lệ ảnh gần trùng tính bằng dHash là ước lượng thô, vì dHash chủ yếu phản ánh nền đường (Hình 3b).

Về phương pháp và đánh giá.

- Chỉ chạy một lần, một hạt giống ngẫu nhiên, nên không có khoảng tin cậy cho các con số.
- Val của B chỉ có 2 video và đông xe hơn train: ước lượng có phương sai cao và lệch về phía khó.
- Tập val của B được dùng cho **cả** dừng sớm, chọn trọng số tốt nhất, chọn ngưỡng **và** báo cáo kết quả, nên con số cuối cùng hơi lạc quan. Đúng ra cần một tập kiểm thử có nhãn, tách riêng.
- A và B được chấm trên hai tập val khác nhau, và B không qua giai đoạn `close_mosaic`, nên chênh lệch A − B không thuần là rò rỉ.
- Không tìm kiếm siêu tham số và không so sánh với mô hình khác: YOLOv8n/m, YOLO11, RT-DETR, hay đơn giản là mô hình COCO gốc với lớp *motorcycle* sẵn có.
- Hai cách tính precision (0,924 và 0,807) chưa được đối chiếu xong.

Về khai phá video và triển khai.

- Mỗi frame được xử lý như một ảnh rời. Chưa có theo vết đối tượng, đếm xe hay phân tích theo thời gian, nên đồ án mới dừng ở mức khai phá **ảnh**, chưa phải khai phá **video** đầy đủ.
- Chưa chạy trên video gốc và chưa đo tốc độ trên phần cứng biên (camera, Jetson…). File ONNX chưa được kiểm tra lại độ chính xác.

5.2. Hướng phát triển

- **Sửa nhãn (ưu tiên số 1):** xuất danh sách FP, rà từng trường hợp, viết **quy tắc gán nhãn** rõ ràng (xe bị cắt ở mép thì có gán không, xe nhỏ tới mức nào vẫn gán) rồi bổ sung nhãn và huấn luyện lại. Đây là cách duy nhất tăng thật cả precision lẫn mAP.
- **Đánh giá chắc chắn hơn:** cross-validation theo video (GroupKFold, ví dụ 5 fold) để báo cáo dạng trung bình ± độ lệch; tách một video làm tập kiểm thử có nhãn; tăng `patience` để B cũng qua giai đoạn `close_mosaic`.
- **So sánh mô hình:** thêm mốc so sánh là mô hình COCO chưa tinh chỉnh, các cỡ n/m, YOLO11 và RT-DETR.
- **Thêm ví dụ khó:** xe đạp, ô tô bị cắt ở mép ảnh, cảnh đêm và cảnh mưa.
- **Khai phá video:** dùng ByteTrack [29] (có sẵn trong Ultralytics qua `model.track`) để gán mã cho từng xe qua các frame, rồi đếm xe qua vạch ảo và vẽ mật độ theo thời gian. Khi đó tri thức rút ra mới là tri thức **về giao thông**, không chỉ về mô hình.

6. Kết luận

Đồ án đã thực hiện trọn một vòng khai phá dữ liệu ảnh cho bài toán phát hiện xe máy: từ khảo sát và kiểm tra chất lượng dữ liệu bằng các công cụ xử lý ảnh cơ bản, làm sạch và chia lại dữ liệu theo video, huấn luyện YOLOv8s bằng học chuyển giao, đến đánh

giá, chọn ngưỡng và phân tích lỗi. Với **CH1**, dữ liệu gốc có rò rỉ (45 nhóm ảnh trùng, 24,3% ảnh valid gần trùng train), có nhiễu nhãn (IoU giữa 2 lần gán nhãn 0,89) và thiếu nhãn. Với **CH2**, trên hai video chưa từng thấy, mô hình đạt mAP@0.5 = 0,980 và mAP@0.5:0.95 = 0,775; rò rỉ chỉ làm điểm đẹp lên chừng 0,01–0,03. Với **CH3**, phần lớn "lỗi báo nhầm" là xe thật bị bỏ sót khi gán nhãn; lỗi thật của mô hình tập trung ở xe nhỏ, ở xa và vật bị cắt ở mép ảnh.

Bài học lớn nhất của đồ án là: **khi điểm số thấp, hãy kiểm tra thước đo trước khi sửa mô hình.** Các bước tiếp theo hợp lý nhất là sửa nhãn, đánh giá bằng cross-validation theo video, và bổ sung theo vết đối tượng để khai thác yếu tố thời gian của video.

Lời cảm ơn. Báo cáo sử dụng bộ dữ liệu công khai trên Roboflow Universe (giấy phép CC BY 4.0), thư viện mã nguồn mở Ultralytics và tài nguyên GPU miễn phí của Kaggle.

Phụ lục. Bảng thuật ngữ

Bảng 10. *Thuật ngữ dùng trong báo cáo*

Thuật ngữ	Giải thích ngắn
Box (bounding box)	Hình chữ nhật bao quanh vật thể, gồm toạ độ tâm, chiều rộng, chiều cao
Ground truth (GT)	Nhãn thật do con người gán, dùng làm "đáp án"
Confidence (điểm tin cậy)	Mức chắc chắn của mô hình rằng trong box có xe (0–1)
IoU	Diện tích giao chia diện tích hợp của hai box; 1 là trùng khít
TP / FP / FN	Dự đoán đúng / báo nhầm / bỏ sót
Precision / Recall / F1	Tỉ lệ đúng trong các dự đoán / tỉ lệ xe thật được tìm thấy / trung bình điều hoà của hai tỉ lệ
AP, mAP@0.5, mAP@0.5:0.95	Diện tích dưới đường P–R; chấm ở IoU 0,5 (dễ) hoặc trung bình 10 mức IoU 0,5–0,95 (khó)
NMS	Bỏ các box chồng lên nhau, chỉ giữ box có điểm cao nhất
Epoch / batch	Một lượt đi qua toàn bộ dữ liệu train / số ảnh xử lý trong một bước cập nhật
Augmentation	Biến đổi ảnh ngẫu nhiên khi huấn luyện để tăng độ đa dạng
Học chuyển giao	Bắt đầu từ trọng số đã học trên bộ dữ liệu lớn (COCO) rồi tinh chỉnh
Backbone / neck / head	Phần trích đặc trưng / phần trộn đặc trưng nhiều tầng / phần đưa ra box và lớp
Anchor-free	Dự đoán box trực tiếp từ mỗi ô, không dùng khung mẫu định sẵn
Rò rỉ dữ liệu	Thông tin của tập kiểm định lọt vào tập huấn luyện, làm điểm đẹp giả tạo
Dừng sớm (early stopping)	Ngừng huấn luyện khi điểm trên val không còn cải thiện
dHash / Hamming	Vân tay 64 bit của ảnh / số bit khác nhau giữa hai vân tay
GroupShuffleSplit	Chia dữ liệu ngẫu nhiên theo nhóm (ở đây là video), nhóm không bị tách
ONNX	Định dạng mô hình trung lập, chạy được ngoài PyTorch

Tài liệu tham khảo

[1] U. Fayyad, G. Piatetsky-Shapiro, and P. Smyth, "From data mining to knowledge discovery in databases," *AI Magazine*, vol. 17, no. 3, pp. 37–54, 1996.

[2] P. Chapman, J. Clinton, R. Kerber, T. Khabaza, T. Reinartz, C. Shearer, and R. Wirth, *CRISP-DM 1.0: Step-by-step data mining guide*, SPSS Inc., 2000.

[3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE CVPR*, 2016, pp. 779–788.

[4] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[5] Roboflow Universe, "motobike-detection (otper-h43mq), version 1," CC BY 4.0. Kaggle mirror: https://www.kaggle.com/datasets/hansrenek/motorbike-detection-yolov8

[6] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in *Proc. IEEE CVPR*, 2001, pp. I-511–I-518.

[7] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in *Proc. IEEE CVPR*, 2005, pp. 886–893.

[8] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Proc. NeurIPS*, 2015, pp. 91–99.

[9] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "SSD: Single shot multibox detector," in *Proc. ECCV*, 2016, pp. 21–37.

[10] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *Proc. ECCV*, 2020, pp. 213–229.

[11] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in *Proc. ECCV*, 2014, pp. 740–755.

[12] C.-Y. Wang, H.-Y. M. Liao, Y.-H. Wu, P.-Y. Chen, J.-W. Hsieh, and I.-H. Yeh, "CSPNet: A new backbone that can enhance learning capability of CNN," in *Proc. IEEE CVPR Workshops*, 2020, pp. 390–391.

[13] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proc. IEEE CVPR*, 2017, pp. 2117–2125.

[14] S. Liu, L. Qi, H. Qin, J. Shi, and J. Jia, "Path aggregation network for instance segmentation," in *Proc. IEEE CVPR*, 2018, pp. 8759–8768.

[15] X. Li, W. Wang, L. Wu, S. Chen, X. Hu, J. Li, J. Tang, and J. Yang, "Generalized focal loss: Learning qualified and distributed bounding boxes for dense object detection," in *Proc. NeurIPS*, 2020, pp. 21002–21012.

[16] C. Feng, Y. Zhong, Y. Gao, M. R. Scott, and W. Huang, "TOOD: Task-aligned one-stage object detection," in *Proc. IEEE ICCV*, 2021, pp. 3490–3499.

[17] Z. Zheng, P. Wang, W. Liu, J. Li, R. Ye, and D. Ren, "Distance-IoU loss: Faster and better learning for bounding box regression," in *Proc. AAAI*, 2020, pp. 12993–13000.

[18] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?" in *Proc. NeurIPS*, 2014, pp. 3320–3328.

[19] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, "The PASCAL visual object classes (VOC) challenge," *Int. J. Comput. Vis.*, vol. 88, no. 2, pp. 303–338, 2010.

[20] J. L. Pech-Pacheco, G. Cristóbal, J. Chamorro-Martínez, and J. Fernández-Valdivia, "Diatom autofocusing in brightfield microscopy: A comparative study," in *Proc. ICPR*, 2000, vol. 3, pp. 314–317.

[21] N. Krawetz, "Kind of like that," *The Hacker Factor Blog*, 2013. [Online]. Available: https://www.hackerfactor.com/blog/index.php?/archives/529-Kind-of-Like-That.html

[22] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Trans. Knowl. Discov. Data*, vol. 6, no. 4, pp. 1–21, 2012.

[23] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[24] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal speed and accuracy of object detection," arXiv:2004.10934, 2020.

[25] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, "mixup: Beyond empirical risk minimization," in *Proc. ICLR*, 2018.

[26] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. ICLR*, 2019.

[27] I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," in *Proc. ICLR*, 2017.

[28] C. G. Northcutt, A. Athalye, and J. Mueller, "Pervasive label errors in test sets destabilize machine learning benchmarks," in *Proc. NeurIPS Datasets and Benchmarks Track*, 2021.

[29] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, "ByteTrack: Multi-object tracking by associating every detection box," in *Proc. ECCV*, 2022, pp. 1–21.